

CS4094 MS2 Progress Report: Distributed DAG Task Scheduler

Revised specification: [specification.md](#). Numeric results are attributed to the committed evidence under `results/handoffs/`; per-step records are in `docs/handoffs/`. The [claim-to-test matrix](#) links the supported claims to their checks. Steps 0-13 are accepted at `main @ bf43c619d00ee1658c5cb1eb297a5e1f59a2de8d`; this report is the Step 14 documentation close-out.

A. What works

All planned MS2 functionality works on the happy path.

Submission. A client submits a DAG as a versioned JSON manifest. The scheduler rejects cycles, unknown or duplicate parents, bad input bindings, unsupported operations, and jobs over the limits (128 tasks, 512 edges, 1 MiB manifest, 32 MiB per file).

Scheduling. Accepted jobs run under a single scheduler mutex. Ready tasks are handed out in FIFO order, independent branches run in parallel on different workers, joins wait for every parent, and a job succeeds only when all of its tasks have.

Workers. Workers poll for work, must be acknowledged before starting, run one task at a time from a fixed list of operations, and exchange files only through the immutable artifact store. A reported failure is retried with a higher attempt number. Replayed messages are answered from cached receipts, and stale or conflicting reports are rejected.

Workloads. The five-task functional DAG produces `result=22`. The video DAG turns a committed 10-second synthetic clip into 720p and 360p MP4s, a PNG thumbnail, fixture subtitles and a publish manifest.

Tooling. Service-backed pytest tests export event histories, metrics and snapshots, and an offline checker re-verifies the safety properties from those files. Compose services are capped at 1 CPU / 512 MiB each. `make build / up / demo / test / fault-demo / bench / down` provide the evaluator flow documented in [README.md](#). The test and fault-demo targets use a pinned Linux harness container; build uses a host JDK and Maven, and demo/benchmark use host Python.

B. Repository layout

Path	Contents
protocol/, scheduler/, artifact-store/, worker/, client/	Java 21 Maven modules
tests/unit, tests/integration, tests/faults, tests/harness	pytest suites, deployment adapters (native, Compose, Hokea), history checker, demo driver
benchmarks/	Benchmark driver and report generator
deploy/	Compose, the test container, the Hokea adapter
workloads/	Functional and video workloads with expected outputs and checksums
docs/	Architecture, roadmap, specification, API, evidence format, this report, step handoffs
results/handoffs/step-XX/	Committed evidence for each roadmap step

C. Tests and results

The suite contains 61 JUnit tests (wire contract, scheduler state machine and artifact store over real HTTP; retained counts in `results/handoffs/step-10/checks/focused-java/command.log`). Step 12 records `mvn -B verify` exit 0 and 86 pytest passes plus one expected failure after the history-checker fix.

Step 13 independently verified tested revision `f27aec9e1230b07c191f34f6a2278eafb0351f4f` from a fresh Ubuntu checkout: `make test` **collected 87 pytest tests, with 86 passed, exactly 1 intentional XFAIL and 0 failures**. The XFAIL is the crash-reassignment oracle. All **40/40 generated evidence directories** passed `check_evidence`, including the real-failure crash run. No implementation changes were required. The committed [Step 13 verification record](#) attributes these results to Ubuntu 24.04.4 LTS, Docker 29.7.2, Compose v5.5.1 and Python 3.12.3; its test counts are not a new JUnit count.

We found and fixed six defects during verification. Each has a test that fails on the old code.

1. An incomplete input binding crashed manifest validation, and the server returned 503 instead of 400.
2. Artifact keys accepted non-canonical UUIDs such as `1-1-1-1-1`.
3. Re-uploading identical bytes with a different media type was silently accepted.
4. A worker process exited whenever the scheduler rejected one of its requests, for example a stale report.
5. The history checker flagged a false violation when a running job's snapshot was compared with a later, longer history. It now compares only against the history up to the snapshot.
6. One Hokea unit test failed on Windows only, because it wrote its fixture with Windows line endings.

D. The intentional failure

The property we leave broken is MS1's second liveness claim: a failed worker's unfinished task eventually becomes available for reassignment. Fixing it means telling a slow worker from a dead one and expiring ownership safely, which is the core of MS3. Leaving it broken costs no safety.

`tests/faults/test_worker_crash.py::test_worker_crash_reassignment` works like this:

1. Worker A takes task X of job X→Y and pauses inside the operation, after the scheduler has acknowledged its start but before it writes any output.
2. The test kills A with SIGKILL.
3. A healthy worker B starts. It completes an unrelated probe job, which shows the system is otherwise live, and keeps asking for work.
4. The test waits 10 seconds for X to be reassigned to B.

In MS2 that never happens: X stays RUNNING under the dead worker, Y stays BLOCKED, and the job stays RUNNING. The cause is structural. The only ways out of a RUNNING attempt are reports from its owner, and the owner is dead. The test also checks that safety holds throughout: X is never falsely completed, Y never starts early, and X never succeeds twice.

`make test` runs this as one strict expected failure that accepts only the `RecoveryNotObserved` exception. Any other error, or an unexpected pass, fails the build. `make fault-demo` runs the same oracle with `--runxfail` and exposes a real nonzero failure. Step 13 confirmed that its only failure was `RecoveryNotObserved`: X remained RUNNING under killed A instead of being reassigned to healthy B. The exact Step 13 shell exit code was not preserved and is not claimed. The 10 seconds is an observation budget, not a bound on recovery time. The task and job remain RUNNING; neither is described as failed or completed.

E. Performance (Step 12)

We ran every combination of $C = 1, 4, 16$ concurrent jobs and $W = 1, 2, 4$ one-task-at-a-time workers, with five fresh deployments each: 45 runs in total.

Each run used a new Compose deployment capped at 1 CPU / 512 MiB per service. It ran 4 warmup jobs, which we discarded, then 24 measured six-task jobs, where each task is a 100 ms fixture operation. A closed loop kept at most C jobs in flight.

All three metrics come from the scheduler's own monotonic clock:

- **Job time:** acceptance to completion.
- **Scheduling latency:** ready to assigned.
- **Throughput:** completed tasks per second over the measured interval.

All 45 runs completed (1,080 jobs and 6,480 tasks). Every output was correct, and in-flight jobs never exceeded C. Percentiles use the nearest-rank method, pooled over the five runs.

Throughput is the mean (standard deviation) of the five per-run values.

C	W	Job time p50 / p95 (ms)	Scheduling p50 / p95 (ms)	Throughput (tasks/s)
1	1	1,398 / 1,454	5.7 / 450	4.20 (0.02)
1	2	1,149 / 1,202	4.4 / 228	5.09 (0.06)
1	4	961 / 1,027	4.9 / 80	6.04 (0.10)
4	1	5,435 / 5,543	1,107.4 / 2,251	4.34 (0.01)
4	2	2,800 / 2,921	459.3 / 1,133	8.31 (0.04)
4	4	1,488 / 1,570	81.5 / 469	15.36 (0.04)
16	1	19,561 / 21,889	3,934.4 / 9,731	4.35 (0.06)
16	2	9,951 / 11,122	1,881.0 / 4,785	8.56 (0.03)
16	4	5,151 / 5,718	834.2 / 2,329	16.69 (0.28)

Step 13 also completed a separate **c1-w1, one-repetition Compose reproduction** with 24 completed jobs, 3.6789568629381906 tasks/s, job p50 1605.221847 ms and scheduling p50 24.604612 ms. Those independent-host observations are in the Step 13 verification record; they are not pooled into the Step 12 matrix.

Workers are the bottleneck. Each one-slot worker completes about 4.2 tasks per second, so once there is enough work ($C \geq 4$), throughput roughly doubles with each doubling of workers. At $C = 1$ a single job only ever has three tasks that can run at once, so extra workers help less: 5.1 and 6.0 tasks/s instead of 8 and 16.

The scheduler keeps up. When a worker is idle, it gets a ready task in about 5 ms. The large scheduling latencies in the table are tasks waiting for a busy worker, and they grow with C/W as queueing predicts. Task execution time stayed at 172-188 ms from $C = 1$ to $C = 16$.

Where a task's time goes. The following approximate cost breakdown is an interpretation of the Step 12 timings, not separately instrumented component measurements. Each task takes roughly 240 ms on a worker:

- 100 ms for the operation itself;
- 75-90 ms moving and verifying files over HTTP;
- about 45 ms for the start acknowledgment.

Repetitions were very consistent: the throughput standard deviation was at most 0.28 tasks/s.

Test environment. All runs were on one Windows 11 machine with 12 CPUs and 16 GB of RAM, using Docker Desktop 27.1.1. We capped its Linux VM at 3 GB, and host free memory stayed above 1.9 GB throughout.

Two earlier attempts without that cap starved the host: free memory fell to 0.4-1 GB, services stalled together, and Docker crashed. We kept those runs as aborted records and did not use them.

Raw data, the full method and the generated tables are in `results/handoffs/step-12/full-20261003c/` and `docs/handoffs/step-12.md`. We promised no performance target and claim none. Recovery time is not measured, because MS2 cannot recover crashed workers' tasks.

F. Deployment

Local. `make up` starts one scheduler (port 8080), one artifact store (port 8081) and `WORKERS` workers under Docker Compose. Containers never restart automatically, so a killed worker stays dead for the crash test. Step 13 verified `make up`, `make demo` and successful `make down`: the functional output was `result=22`; `video720.mp4` was H.264 at 1280x720, `video360.mp4` was H.264 at 640x360, and `thumbnail.png` was PNG at 1280x720. Fixture subtitles were generated and demo history checks passed.

Course cluster. The Hokea adapter (Step 11) is pinned to Hokea revision `427b94634b1736ba8e59d4977836162aa58bd2cb` and passed local Hokea, Compose and Make verification: `make test 84 passed + 1 xfailed`, and `make fault-demo` with only the intended failure.

We also attempted a run on the course cluster (namespace `team-06`, public GHCR images). Hokea started the runner and `pytest`, but all three tests stopped during setup, before any of our services ran, because the Hokea package installed in the course runner is not the pinned revision. We did not loosen the pin to work around this. The run can be repeated unchanged once course staff provide a matching runner. Evidence:

`results/handoffs/step-11/acceptance-hokea-20261003T200759-f99f027a/`.

G. Limitations

- Crashed workers' tasks are never recovered (Section D). This is intentional for MS2.
- Scheduler state lives in memory. A scheduler restart begins a new, empty run.
- The artifact store's index lives only as long as its process. The files stay on disk, but a restarted store does not rebuild its index from them.
- Workers are trusted; their results are not recomputed.
- Failed operations are retried forever, so an operation that always fails leaves its job running.
- Benchmark numbers come from one shared Windows host with a synthetic workload. They include polling delay, depend on the host's memory, and do not cover recovery time.
- Course-cluster service behavior remains unverified: execution was attempted and blocked during setup by the external runner mismatch (Section F).
- Independent clean-checkout verification (Step 13) passed the normal suite, intentional recovery failure, functional/media demo, all 40 evidence-directory checks and the small benchmark reproduction without implementation changes. A transient `java.io.IOException: connection closed before all data received` harness diagnostic caused no test failure; the exact `fault-demo` shell exit code was not retained. This verification does not establish course-cluster execution or recovery support.

H. Deviations

There are no architecture deviations. Every behavior change was a bug fix toward the contract: the six defects in Section C, plus a missing run ID now returning 400 instead of 409.

Within the architecture's test-container role, we made three additions. The container runs tests on its own copy of the code, because a Windows/OneDrive mount made the JVM too slow. The video sample is regenerated by a committed, byte-reproducible script, so its provenance is exact. And `.gitattributes` keeps checksummed files byte-identical across operating systems.

Step 12 was developed on a separate branch while Step 11 was being finished. It merged the accepted `main` before measuring, and the measured service code is identical to what passed at Step 9.

I. MS3 handoff

The approved MS3 extension is scheduler-side leases on the existing injected monotonic clock; none are implemented in MS2. When a lease expires, the attempt is marked expired and the task goes back to the end of the ready queue.

Most of the groundwork already exists:

- owner and attempt-number checks already reject late results;
- per-attempt output paths already prevent overwrites;
- the event schema and history checker carry over unchanged.

With leases in place, we remove the expected-failure marker so the crash test must pass, and add tests for crashes, pauses and lost reports at each stage of an attempt. Scheduler durability, replication and exactly-once external effects remain separate decisions.